Pixel and Bit - Technical Documentation

Generated 2026-04-22 12:53 UTC

Pixel & Bit — Technical Documentation

> This document is the single entry point into the project. It points to five focused files under `/docs/` that together cover everything: architecture, every backend class, every Razor view/page, the database schema, authentication, deployment, and a study plan.

Product: Pixel & Bit — a tech-repair shop website in Jordan. It offers:

- Device repair **booking** with a public **ticket tracking** system (e.g. `PB-2026-1A2B`)
- An **online store** (service/product catalog) with a **session-based cart** and simple checkout
- An **admin dashboard** (products, orders, bookings, users)
- **ASP.NET Core Identity** authentication with a custom **5-digit email verification** flow and a custom **multi-step forgot-password** flow
- **Bilingual UI** (English / Arabic RTL) via `IStringLocalizer` + a cookie-based culture switcher

Stack

- **.NET 8** (ASP.NET Core MVC + Razor Pages Identity UI)
- **Entity Framework Core 8** with **SQLite** (file: `pixelbit.db`)
- **ASP.NET Core Identity** (`IdentityUser`, `IdentityRole`)
- **MailKit / MimeKit** for SMTP email
- **Bootstrap 5** + custom `pixelbit.css` (dark premium theme, gradients, glass)
- Session-backed cart + password-reset state; distributed memory cache

Solution layout (4 projects, Clean-Architecture-ish):

```
PixelAndBit.sln
├── PixelAndBit.Domain          ← entities + enums (no dependencies)
├── PixelAndBit.Application     ← service interfaces + DTOs
├── PixelAndBit.Infrastructure  ← EF Core, DbContext, services, email, migrations
└── PixelAndBit.Web             ← MVC controllers, Razor Pages, views, wwwroot
```

Documentation map

#	File	What it covers
1	`01_Overview.md`	Project overview, architecture, folder tree, request flow, `Program.cs` walkthrough
2	`02_Backend.md`	Every controller + action, every service, every interface, every entity/enum/view-model, Identity pages
3	`03_Frontend.md`	Razor views, layout, navbar, auth UI, CSS design system, `site.js`, localization + RTL
4	`04_Database_Auth_Config.md`	`PixelBitDbContext`, entity configurations, migrations, schema tables, auth/authorization, `appsettings`, SMTP
5	`05_Deployment_and_Study.md`	Running locally, publishing, Linux/Kestrel/systemd/Nginx patterns, study plan, risks & improvements

---

Quick start (TL;DR)

```
# From the repo root, in a dedicated terminal:
./scripts/start-dev.ps1
# → http://localhost:5001
```

Development seeds an admin: `admin@pixelbit.jo` / `Admin@Pb2026!`
(see `PixelAndBit.Infrastructure/Data/DbSeeder.cs`).

Documentation file created at: `/docs/PixelAndBit_Documentation.md`

01 — Project Overview & Architecture

1. What Pixel & Bit is

Pixel & Bit is a small-business web application for a **tech-repair shop** in Amman, Jordan. It combines four things in one ASP.NET Core 8 web app:

1. **Booking** — a customer fills a form ("name, phone, device, issue") and receives a **unique ticket reference** (`PB-2026-XXXX`). They can come back any time and **track** the status.
2. **Store** — a public catalog of services/products. Visitors add to a **cart** (kept in server session, not DB) and confirm an **order**.
3. **Admin** — an authenticated admin with the `Admin` role manages products, views/edits orders, watches a live dashboard (totals + revenue), inspects users, and moves booking statuses forward.
4. **Accounts** — ASP.NET Core Identity with:

- Email + password login
- Public registration gated by a single `AllowPublicRegistration` flag
- Custom **5-digit email verification** code flow (`EmailVerificationCodes` table)
- Custom **multi-step forgot-password** flow (session-backed, reuses Identity's `GeneratePasswordResetTokenAsync`)
- Role-based authorization (`Admin` role) on the admin surface

All UI text is **localized (en / ar)**. Arabic renders RTL with Bootstrap RTL + custom overrides. The visual theme is a dark "premium" look (glassmorphism, gradient accents, neon glow buttons) — one shared stylesheet: `wwwroot/css/pixelbit.css`.

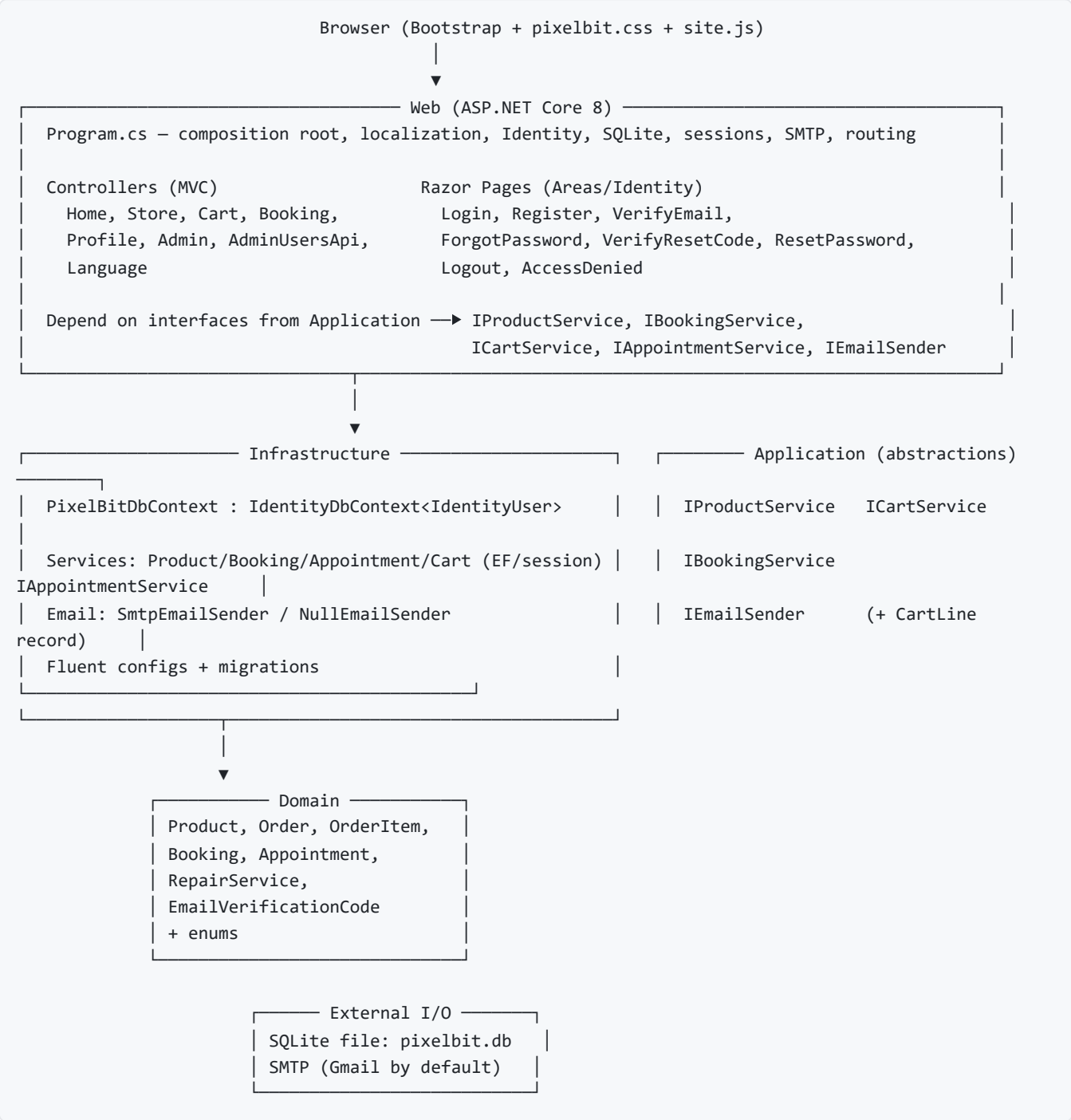
---

2. Solution / folder tree

```
PixelAndBit/
├── PixelAndBit.sln
├── README_ADMIN.md                     # Operator cheat-sheet
├── scripts/
│   ├── start-dev.ps1                   # dotnet watch run — keep window open
│   └── start-dev.bat
├── tools/PixelAndBit.ImageTool/        # Helper (image processing; not in Web)
├── publish*/                           # Old dotnet publish outputs (deploy artifacts)
│
├── PixelAndBit.Domain/                 # Pure C# (no ASP.NET/EF deps)
│   ├── Entities/
│   │   ├── Product.cs
│   │   ├── Order.cs
│   │   ├── OrderItem.cs
│   │   ├── Booking.cs
│   │   ├── Appointment.cs
│   │   ├── RepairService.cs
│   │   └── EmailVerificationCode.cs
│   └── Enums/
│       ├── OrderStatus.cs
│       ├── BookingStatus.cs
│       ├── AppointmentStatus.cs
│       └── ProductCondition.cs
│
├── PixelAndBit.Application/            # Abstractions only
│   └── Interfaces/
│       ├── IProductService.cs
│       ├── ICartService.cs  (defines CartLine record)
│       ├── IBookingService.cs
│       ├── IAppointmentService.cs
│       └── IEmailSender.cs
│
├── PixelAndBit.Infrastructure/         # EF Core + implementations
│   ├── Data/
│   │   ├── AppDbContext.cs             # PixelBitDbContext : IdentityDbContext<IdentityUser>
│   │   ├── DbSeeder.cs                 # Roles, admin user, seed products
│   │   ├── ProductService.cs           # IProductService
│   │   ├── BookingService.cs           # IBookingService + ticket generator
│   │   ├── AppointmentService.cs       # IAppointmentService (partial)
│   │   ├── CartService.cs              # ICartService (session-backed)
│   │   └── Configurations/             # Fluent EF per-entity configs
│   │       ├── ProductConfiguration.cs
│   │       ├── OrderConfiguration.cs
│   │       ├── OrderItemConfiguration.cs
│   │       ├── BookingConfiguration.cs
│   │       ├── AppointmentConfiguration.cs
│   │       ├── RepairServiceConfiguration.cs
│   │       └── EmailVerificationCodeConfiguration.cs
│   ├── Email/
│   │   ├── SmtpEmailSender.cs          # MailKit — real sender
│   │   └── NullEmailSender.cs          # Logs only; used when SMTP not configured
│   └── Migrations/
│       ├── 20260418124033_InitialSqlite.cs
│       └── PixelBitDbContextModelSnapshot.cs
│
└── PixelAndBit.Web/                    # The ASP.NET Core host
    ├── Program.cs                      # Composition root
```

```
├── PixelAndBit.Web.csproj          # TFM: net8.0
├── appsettings.json / *.Development.json / *.Production.json
├── pixelbit.db                     # SQLite file (dev)
│
├── Controllers/                    # 8 controllers (see 02_Backend.md)
│   ├── HomeController.cs
│   ├── StoreController.cs
│   ├── BookingController.cs
│   ├── CartController.cs
│   ├── ProfileController.cs
│   ├── AdminController.cs
│   ├── AdminUsersApiController.cs  # JSON API under /api/admin
│   └── LanguageController.cs
│
├── Models/                         # ViewModels + DTOs
│   ├── CreateBookingVm.cs
│   ├── TrackBookingVm.cs
│   ├── AdminDashboardVm.cs
│   ├── AdminProductEditVm.cs
│   └── ErrorViewModel.cs
│
├── Resources/                      # Localization
│   ├── SharedResource.cs           # marker class
│   ├── SharedResource.resx         # English strings
│   └── SharedResource.ar.resx      # Arabic strings
│
├── Areas/Identity/Pages/Account/   # Razor Pages for auth
│   ├── Login.cshtml(.cs)
│   ├── Register.cshtml(.cs)
│   ├── Logout.cshtml(.cs)
│   ├── AccessDenied.cshtml(.cs)
│   ├── RegisterConfirmation.cshtml(.cs)
│   ├── VerifyEmail.cshtml(.cs)     # 5-digit email verify
│   ├── ForgotPassword.cshtml(.cs)  # Reset — step 1
│   ├── VerifyResetCode.cshtml(.cs) # Reset — step 2
│   ├── ResetPassword.cshtml(.cs)   # Reset — step 3
│   └── PasswordResetFlow.cs        # session keys + helpers + reset email HTML
│
├── Views/
│   ├── _ViewImports.cshtml, _ViewStart.cshtml
│   ├── Shared/
│   │   ├── _Layout.cshtml
│   │   ├── _Navbar.cshtml
│   │   ├── Error.cshtml
│   │   └── _ValidationScriptsPartial.cshtml
│   ├── Home/       Index, Privacy, Contact
│   ├── Store/      Index
│   ├── Booking/    Index, Track, Success, Admin
│   ├── Cart/       Index, Success
│   ├── Profile/    Index, MyOrders, MyRepairs
│   └── Admin/      Index, Users, Products, ProductCreate, ProductEdit, Orders, OrderDetails
│
└── wwwroot/
    ├── favicon.ico, pixelbit-nav-icon.png
    ├── css/   site.css, pixelbit.css (large, ~2800 lines), tailwind.bundle.css
    ├── js/    site.js (fade-in, parallax, navbar scroll, device chips, cart AJAX)
    └── lib/   bootstrap, jquery, jquery-validation(-unobtrusive)
```

3. Architecture at a glance



Key dependency rule: `Web` → `Application` + `Infrastructure` → `Domain`.

- `Domain` has no dependencies.
- `Application` only depends on `Domain`.
- `Infrastructure` implements the interfaces from `Application` using EF Core + `Domain` entities.

- `Web` is the only layer that references everything and wires DI in `Program.cs`.

---

4. Typical request flow

4.1 "Book a repair" (happy path)

1. Visitor browses `/Booking` → returns `Views/Booking/Index.cshtml` bound to `CreateBookingVm`.
2. Form posts to `BookingController.Confirm(vm)` with `@Html.AntiForgeryToken()`.
3. Controller calls `IBookingService.CreateBookingAsync(...)` → `BookingService` validates (Jordanian phone `^07\d{8}$`, description ≥ 20 chars, etc.), generates `PB-YYYY-XXXX` (base-36, retries on collision), persists `Booking`, returns `(ok, ticket, err)`.
4. Controller redirects to `Booking/Success?ticket=PB-2026-1A2B`, and the success view shows the ticket for the customer to save.

4.2 "Track my repair"

1. Visitor hits `/Booking/Track`, submits a ticket reference.
2. `BookingController.Track(vm)` calls `IBookingService.GetByTicketAsync(...)` → uppercased lookup against `Bookings.TicketReference` (unique index).
3. The view renders the current `BookingStatus` badge; if not found, it adds a model error.

4.3 "Shop + checkout" (anonymous)

1. `/Store` → `StoreController.Index` lists all products and supports `?q=` simple search (`Name` / `Description`, case-insensitive).
2. Each card has a POST button → `CartController.Add([FromBody] AddToCartRequest)` (AJAX). `CartService` stores `Dictionary<int,int>` in **session** under `pb_cart_v1`; returns `{count, total}`.
3. `/Cart` → `CartController.Index` re-hydrates lines from the DB by product IDs.
4. `Confirm` creates a real `Order` + `OrderItem[]` in DB, clears the session cart, redirects to `/Cart/Success/{id}`.

4.4 Sign in / Register / Verify

1. `/Identity/Account/Register` (enabled by `AllowPublicRegistration=true`) creates the `IdentityUser` with `EmailConfirmed = false`.
2. A random 5-digit code is generated, salted-hashed (`SHA256(userId|EMAIL|code)`), and persisted in `EmailVerificationCodes` with a 10-minute TTL + 6-attempt cap.
3. User is redirected to `/Identity/Account/VerifyEmail?email=...`. JS auto-submits when 5 digits are entered.
4. Correct code → `user.EmailConfirmed = true` + `SignInAsync`.
5. `/Identity/Account/Login` is the shared form for users and admins; after login, if the user has the `Admin` role, the navbar shows the Admin link and `[Authorize(Roles="Admin")]` permits `/Admin/*`.

4.5 Forgot password (multi-step, session-backed)

See `Areas/Identity/Pages/Account/PasswordResetFlow.cs`:

- **Step 1 /ForgotPassword** — always stores the email in session (even when not found), generates code + Identity `GeneratePasswordResetTokenAsync`, sends an email, redirects to Step 2.
- **Step 2 /VerifyResetCode** — gated on `HasPendingEmail`; `SHA256("reset|userId|EMAIL|code")` + `FixedTimeEquals`; attempts counter incremented before comparison; after success sets `verified=1`, removes the code hash.
- **Step 3 /ResetPassword** — gated on `IsCodeVerified`; calls `UserManager.ResetPasswordAsync(user, token, newPw)` + `UpdateSecurityStampAsync`; wipes session state; redirects to Login with toast.

---

5. `Program.cs` — composition root walkthrough

File: `PixelAndBit.Web/Program.cs`

5.1 Kestrel binding

```
if (string.IsNullOrWhiteSpace(Environment.GetEnvironmentVariable("ASPNETCORE_URLS")))
{
    if (builder.Environment.IsDevelopment())
        builder.WebHost.UseUrls("http://0.0.0.0:5001"); // LAN + localhost for mobile testing
    else
        builder.WebHost.UseUrls("http://127.0.0.1:5001"); // loopback only; reverse-proxy terminates
TLS
}
```

You can override with `ASPNETCORE_URLS`. Production is loopback-only on purpose: deploy behind Nginx.

5.2 SQLite path + config override

```
var sqlitePath = Path.Combine(builder.Environment.ContentRootPath, "pixelbit.db");
builder.Configuration.AddInMemoryCollection(new Dictionary<string, string?>
{
    ["ConnectionStrings:PixelBitConnection"] = $"Data Source={sqlitePath}"
});
```

The DB file always sits next to the executable regardless of where the process was launched from.

5.3 Logging

Clears providers, adds `SimpleConsole` (with timestamp) + Debug, default level `Information`.

5.4 MVC, Razor Pages, localization

```
builder.Services.AddLocalization();
builder.Services.AddControllersWithViews()
    .AddViewLocalization()
    .AddDataAnnotationsLocalization(o =>
        o.DataAnnotationLocalizerProvider = (_, f) => f.Create(typeof(SharedResource)));
builder.Services.AddRazorPages();
```

`SharedResource.cs` is an empty marker class; the localizer resolves strings from `Resources/SharedResource.resx` (and `.ar.resx`).

5.5 HTTP infrastructure

- `AddHttpContextAccessor()` — so `CartService` and `PasswordResetFlow` can read session.
- `AddDistributedMemoryCache()` — backing store for the session.
- `AddResponseCompression()` — Brotli + Gzip for HTTPS.
- `AddSession()` — cookie name `.PixelAndBit.Session`, `HttpOnly=true`, `IsEssential=true`, `IdleTimeout=6h`.

5.6 Application services

```
builder.Services.AddScoped<IProductService,     ProductService>();
builder.Services.AddScoped<IBookingService,     BookingService>();
builder.Services.AddScoped<IAppointmentService, AppointmentService>();
builder.Services.AddScoped<ICartService,        CartService>();
```

5.7 Email sender — conditional on SMTP config

```
builder.Services.Configure<SmtpEmailOptions>(builder.Configuration.GetSection("Smtp"));
var smtpConfigured = !string.IsNullOrWhiteSpace(smtpHost) &&
                     !smtpHost.Contains("YOUR_", ...) &&
                     !string.IsNullOrWhiteSpace(smtpFrom) && ...;
if (smtpConfigured) services.AddScoped<IEmailSender, SmtpEmailSender>();
else                services.AddSingleton<IEmailSender, NullEmailSender>();
```

This is why the site starts even without SMTP configured: email calls become no-ops.

5.8 EF Core + Identity

```
builder.Services.AddDbContext<PixelBitDbContext>(opts =>
    opts.UseSqlite(sqliteConnectionString,
        sql => sql.MigrationsAssembly("PixelAndBit.Infrastructure")));

builder.Services.AddIdentity<IdentityUser, IdentityRole>(o =>
{
    o.Password.RequireDigit = false;
    o.Password.RequiredLength = 6;
    o.Password.RequireNonAlphanumeric = false;
    o.Password.RequireUppercase = false;
    o.SignIn.RequireConfirmedAccount = true; // email verification enforced
})
.AddEntityFrameworkStores<PixelBitDbContext>()
.AddDefaultTokenProviders()
.AddDefaultUI();
```

5.9 Cookie/auth UX

```
builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Identity/Account/Login";
    options.AccessDeniedPath = "/Identity/Account/AccessDenied";
    options.Events = new CookieAuthenticationEvents
    {
        OnRedirectToLogin        = ctx => /* API paths → 401 instead of redirect */,
        OnRedirectToAccessDenied = ctx => /* API paths → 403 instead of redirect */
    };
});
```

Browsers see HTML redirects; `/api/*` callers get proper HTTP status codes — important for `AdminUsersApiController`.

5.10 Lazy DB initialization

```
Task? dbInitTask = null;
Task EnsureDatabaseInitializedAsync() { /* migrate + (Development) seed, run-once */ }
app.Use(async (ctx, next) => { await EnsureDatabaseInitializedAsync(); await next(); });
```

Hot-reload-friendly: if `dotnet watch` doesn't restart the process, the DB still migrates on the next request. `/__health/db` returns `{ ok, products, database, file }`.

5.11 Pipeline order

```
UseRequestLocalization → (DbInit middleware) →
(Dev) UseDeveloperExceptionPage | (Prod) UseExceptionHandler("/Home/Error") →
UseStaticFiles → UseRouting → UseResponseCompression → UseSession →
UseAuthentication → UseAuthorization →
MapGet("/__health/db") → MapControllerRoute(default "{controller=Home}/{action=Index}/{id?}") →
MapControllers → MapRazorPages
```

5.12 Graceful port-in-use handling

```
catch (Exception ex) when (IsAddressAlreadyInUse(ex)) { log "port busy"; throw; }
```

---

6. Project-level design notes

- **Clean boundary**: controllers never `new DbContext()`. They always go through the service interfaces (`IProductService`, `IBookingService`, `ICartService`, `IAppointmentService`). The notable exceptions are `ProfileController` and `AdminController`, which query the `PixelBitDbContext` directly because those are read-heavy admin/reporting queries.
- **Session over DB for cart**: the cart is not a DB entity. It's a `Dictionary<int, int>` serialized as JSON in session (`pb_cart_v1`). This keeps anonymous checkout frictionless.
- **Ticket generation**: `Booking.TicketReference` is unique; `BookingService.GenerateTicketReferenceAsync` uses secure RNG + base-36 with retry loop to avoid collisions, fallback to a GUID-based suffix.

- **Verification codes are hashed**: raw codes never hit the DB. `HashCode(userId, email, code) = SHA256Hex(userId|EMAIL_UPPER|code)`; comparison uses `CryptographicOperations.FixedTimeEquals` for constant time. Reset codes use the same pattern with a `"reset|"` purpose marker to isolate hash spaces.
- **No custom password tokens**: the actual password mutation uses Identity's own `GeneratePasswordResetTokenAsync` + `ResetPasswordAsync`. Our 5-digit code is only a user-facing gate that unlocks that built-in token stored in session.
- **Localization is centralized**: one resource (`SharedResource.resx`). Controllers and views use `@inject IStringLocalizer<SharedResource> T`. Culture is cookie-based via `LanguageController.Set`.
- **Bidi support**: `_Layout.cshtml` picks the RTL Bootstrap bundle when culture is `ar`; `pixelbit.css` has `.pb-rtl` overrides.

Continue to `02_Backend.md` for the per-class file-by-file tour.

02 — Backend: Controllers, Services, Interfaces, Entities, Identity Pages

This file walks every backend file (C# side). Sections:

1. **Domain** — entities + enums
2. **Application** — interface contracts
3. **Infrastructure** — services + email + seeder + EF configurations
4. **Web: Controllers** — every controller and every action
5. **Web: ViewModels**
6. **Web: Razor Pages (Identity)** — login / register / verify / reset

---

1. Domain

Pure C# classes under `PixelAndBit.Domain/`. No ASP.NET/EF references. Safe to share.

1.1 Entities

`Entities/Product.cs`

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
    public string Description { get; set; } = "";
    public int StockQuantity { get; set; }
    public string? ImageUrl { get; set; }
}
```

A product/service in the store catalog. `Price` is stored as `decimal(10,2)` (see `ProductConfiguration`). `ImageUrl` is optional.

`Entities/Order.cs`

```
public class Order
{
    public Guid Id { get; set; }
    public string? UserId { get; set; }                  // nullable → guest checkout allowed
    public DateTime OrderDate { get; set; } = DateTime.UtcNow;
    public decimal TotalAmount { get; set; }
    public OrderStatus Status { get; set; } = OrderStatus.Pending;
    public ICollection<OrderItem> Items { get; set; } = new List<OrderItem>();
}
```

`Entities/OrderItem.cs`

```
public class OrderItem
{
    public Guid Id { get; set; }
    public Guid OrderId { get; set; }
    public Order Order { get; set; } = null!;
    public int ProductId { get; set; }
    public Product Product { get; set; } = null!;
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }              // price at time of order (snapshot)
}
```

Uses a **price snapshot** so historical orders stay correct even if the product price changes later.

`Entities/Booking.cs`

```
public class Booking
{
    public Guid Id { get; set; }
    public string TicketReference { get; set; } = "";    // e.g. "PB-2026-1A2B" (unique)
    public string? UserId { get; set; }                  // nullable → guests may book
    public string CustomerName { get; set; } = "";
    public string PhoneNumber { get; set; } = "";        // Jordan mobile format validated
    public string DeviceModel { get; set; } = "";
    public string IssueDescription { get; set; } = "";
    public BookingStatus Status { get; set; } = BookingStatus.Pending;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public decimal EstimatedCost { get; set; }
}
```

`Entities/Appointment.cs`

Schedulable appointment (richer than `Booking` — has date/time slots + linked `RepairService`). Used for future slot-based scheduling; the current UI is booking-centric, not appointment-centric, so `AppointmentService` exposes placeholders for `GetAvailableDatesAsync` / `GetAvailableSlotsAsync`.

`Entities/RepairService.cs`

Catalog of repair service types with `BasePrice` and `DurationMinutes` (used for slot generation). Navigation: `ICollection<Appointment> Appointments`.

`Entities/EmailVerificationCode.cs`

```
public class EmailVerificationCode
{
    public Guid Id { get; set; }
    public string UserId { get; set; } = "";
    public string Email { get; set; } = "";
    public string CodeHash { get; set; } = "";           // SHA-256 hex
    public DateTime CreatedAtUtc { get; set; }
    public DateTime ExpiresAtUtc { get; set; }
    public int Attempts { get; set; }
    public int MaxAttempts { get; set; } = 6;
    public DateTime? ConsumedAtUtc { get; set; }
}
```

One row per issued code. Used by `VerifyEmailModel`. Forgot-password codes do **not** use this table — they live in server session.

1.2 Enums

Enum	Values
`BookingStatus`	`Pending = 0, Received = 1, InProgress = 2, ReadyForPickup = 3, Completed = 4`
`OrderStatus`	`Pending = 0, Completed = 1, Cancelled = 2`
`AppointmentStatus`	`Pending, Confirmed, InProgress, Completed, Cancelled`
`ProductCondition`	`New, Refurbished, Used` (defined but not yet used on `Product`)

All enums with DB persistence are stored as **strings** via `HasConversion<string>()` in Fluent configs, so enum values are readable in SQL and safe against numeric drift.

---

2. Application — abstractions

`Interfaces/IProductService.cs`

```
Task<IEnumerable<Product>> GetAllProductsAsync();
Task<Product?> GetProductByIdAsync(int id);
```

`Interfaces/ICartService.cs`

```
Task AddAsync(int productId, int quantity = 1);
Task RemoveAsync(int productId, int quantity = 1);
Task ClearAsync();
Task<IReadOnlyList<CartLine>> GetLinesAsync();
Task<int>      GetItemCountAsync();
Task<decimal>  GetTotalAsync();

public record CartLine(Product Product, int Quantity);
```

Abstracts the "cart" away from the HTTP session implementation. Any DI target can be swapped.

`Interfaces/IBookingService.cs`

```
Task<(bool Success, string? TicketReference, string? ErrorMessage)> CreateBookingAsync(
    string? userId, string customerName, string phoneNumber,
    string deviceModel, string issueDescription, decimal estimatedCost);

Task<IReadOnlyList<Booking>> GetAllAsync();
Task<Booking?>               GetByTicketAsync(string ticketReference);
Task<bool>                   UpdateStatusAsync(Guid bookingId, BookingStatus status);
```

`Interfaces/IAppointmentService.cs`

```
Task<IEnumerable<RepairService>> GetAllRepairServicesAsync();
Task<IEnumerable<DateTime>>      GetAvailableDatesAsync(int serviceId, int month, int year);
Task<IEnumerable<TimeSpan>>      GetAvailableSlotsAsync(int serviceId, DateTime date);
Task<bool>                       CreateAppointmentAsync(Appointment appointment);
```

Two methods are stubs (return empty) pending the future scheduling UI.

`Interfaces/IEmailSender.cs`

```
Task SendAsync(string toEmail, string subject, string htmlBody);
```

Single method. Two implementations in `Infrastructure`.

---

3. Infrastructure

3.1 `Data/AppDbContext.cs` (class: `PixelBitDbContext`)

```
public class PixelBitDbContext : IdentityDbContext<IdentityUser>
{
    public DbSet<Product> Products => Set<Product>();
    public DbSet<Booking> Bookings => Set<Booking>();
    public DbSet<Order> Orders => Set<Order>();
    public DbSet<OrderItem> OrderItems => Set<OrderItem>();
    public DbSet<EmailVerificationCode> EmailVerificationCodes => Set<EmailVerificationCode>();
    public DbSet<RepairService> RepairServices => Set<RepairService>();
    public DbSet<Appointment> Appointments => Set<Appointment>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);                                    // Identity tables
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(PixelBitDbContext).Assembly);
    }
}
```

- Inherits from `IdentityDbContext<IdentityUser>` so the `AspNet*` tables are created automatically.
- `ApplyConfigurationsFromAssembly` auto-registers every `IEntityTypeConfiguration<T>` in the Infrastructure assembly.

3.2 `Data/Configurations/*`

File	Maps	Notes
`ProductConfiguration.cs`	`Product` → `Products`	`Name` required ≤ 200, `Description` required ≤ 2000, `Price` `decimal(10,2)`, `ImageUrl` ≤ 500
`BookingConfiguration.cs`	`Booking` → `Bookings`	`TicketReference` unique (`HasIndex(...).IsUnique()`), status stored as string, `EstimatedCost` `decimal(10,2)`
`OrderConfiguration.cs`	`Order` → `Orders`	`Status` as string, `TotalAmount` `decimal(10,2)`, cascade delete of `Items`
`OrderItemConfiguration.cs`	`OrderItem` → `OrderItems`	`UnitPrice decimal(10,2)`, FK to `Product` with `OnDelete(Restrict)`
`AppointmentConfiguration.cs`	`Appointment` → `Appointments`	`ConfirmationCode` unique, FK to `RepairService` with `OnDelete(Restrict)`
`RepairServiceConfiguration.cs`	`RepairService` → `RepairServices`	`BasePrice decimal(18,2)`
`EmailVerificationCodeConfiguration.cs`	`EmailVerificationCode` → `EmailVerificationCodes`	Index on `(UserId, Email, ConsumedAtUtc, ExpiresAtUtc)` for the verify-latest query

3.3 `Data/DbSeeder.cs`

Static `SeedAsync(ctx, userManager, roleManager)` called from `Program.cs` only when `app.Environment.IsDevelopment()`.

Steps:

1. **Roles** — creates `"Admin"` and `"Customer"` if missing.
2. **Admin** — ensures `admin@pixelbit.jo` with password `Admin@Pb2026!`; marks `EmailConfirmed = true`; adds to `Admin` role.
3. **Seed products** — if the `Products` table is empty, inserts five curated rows (thermal paste, keyboard modding, console cleaning, HDMI mod, entry-level build).

> **Security note**: seeding is dev-only. In production you should change the admin password immediately and remove or edit the default.

3.4 `Data/ProductService.cs`

```
public async Task<IEnumerable<Product>> GetAllProductsAsync()
    => await _context.Products.ToListAsync();

public async Task<Product?> GetProductByIdAsync(int id)
    => await _context.Products.FirstOrDefaultAsync(p => p.Id == id);
```

No filtering or paging; the store is small. Search is done in `StoreController` in-memory after the `ToList`.

3.5 `Data/BookingService.cs`

`CreateBookingAsync(...)` performs:

- Input trimming + validation:
- `customerName` length ≥ 2
- phone matches `^07\d{8}$` (Jordanian mobile)
- `deviceModel` length ≥ 2
- `issueDescription` length ≥ 20
- `estimatedCost ≥ 0`
- Ticket generation: `GenerateTicketReferenceAsync(year)`:
- Up to **8 retries**, each with a 4-char base-36 suffix from `RandomNumberGenerator.Fill(bytes)` → `"PB-{year}-{SUFFIX}"`, collision-checked against DB.
- Final fallback: `PB-{year}-{GuidNpart}` truncated to 21 chars.
- Persists the `Booking` and returns `(true, ticket, null)`.

`GetByTicketAsync(ticket)` — trims, uppercases, and does `FirstOrDefaultAsync` against the unique `TicketReference` index.

`UpdateStatusAsync(id, status)` — fetches by `Id`, mutates `Status`, saves.

`GetAllAsync()` — returns all bookings `OrderByDescending(CreatedAt)`.

3.6 `Data/AppointmentService.cs`

- `GetAllRepairServicesAsync()` — just `ToListAsync()` on `RepairServices`.
- `CreateAppointmentAsync(appointment)` — sets `ConfirmationCode = "PB-" + Guid.NewGuid()[0..5].ToUpper()`, saves; returns `true` if `SaveChangesAsync() > 0`.
- `GetAvailableDatesAsync` / `GetAvailableSlotsAsync` — intentionally return empty; placeholder for future slot calendar.

3.7 `Data/CartService.cs` (session-backed)

```
private const string SessionKey = "pb_cart_v1";
// Stored value: JSON Dictionary<int productId, int quantity>
```

- Pulls `HttpContext` via `IHttpContextAccessor`.
- `GetCart()` — deserializes JSON; returns empty dict on error.
- `AddAsync / RemoveAsync / ClearAsync` — mutate and save JSON back.
- `GetLinesAsync()` — round-trips product IDs to `_db.Products`, then joins in memory:

```
var ids = cart.Keys.ToArray();
var products = await _db.Products.Where(p => ids.Contains(p.Id)).ToListAsync();
// → List<CartLine(Product, Quantity)>
```

- `GetItemCountAsync / GetTotalAsync` — derived from `GetLinesAsync`.

3.8 `Email/SmtpEmailSender.cs`

- Options class: `SmtpEmailOptions { Host, Port=587, EnableSsl=true, Username, Password, FromEmail, FromName="Pixel & Bit" }`.
- `SendAsync`:
- Builds `MimeMessage` with `TextPart(Html)`.
- Connects with `MailKit`, `timeout 30s`, uses `ConnectAsync(host, port, useSsl:true)`, `AuthenticateAsync`, `SendAsync`, disconnects.
- Logs `=== SMTP DEBUG START / END / ERROR ===` blocks to stdout.
- Rethrows on failure — callers (`VerifyEmailModel`, `ForgotPasswordModel`, etc.) wrap with `try/catch` and log without leaking details.

3.9 `Email/NullEmailSender.cs`

```
public Task SendAsync(string toEmail, string subject, string htmlBody)
{
    _logger.LogDebug("Email not sent (SMTP not configured). To={To} Subject={Subject}", toEmail,
subject);
    return Task.CompletedTask;
}
```

Registered when SMTP config is missing — keeps the app usable for local dev without real credentials.

3.10 Migrations

- `Migrations/20260418124033_InitialSqlite.cs` — initial migration creating:
- Identity tables (`AspNetUsers`, `AspNetRoles`, `AspNetUserRoles`, `AspNetUserClaims`, `AspNetUserLogins`, `AspNetRoleClaims`, `AspNetUserTokens`)
- Business tables (`Bookings`, `Products`, `Orders`, `OrderItems`, `Appointments`, `RepairServices`, `EmailVerificationCodes`)
- All indexes (including unique `TicketReference` and the `EmailVerificationCodes` composite index)
- `PixelBitDbContextModelSnapshot.cs` — EF state file.

Migration is applied automatically on first request via `EnsureDatabaseInitializedAsync()` in `Program.cs`.

---

4. Web — Controllers

All controllers live in `PixelAndBit.Web/Controllers`. Default route: `"{controller=Home}/{action=Index}/{id?}"`.

4.1 `HomeController.cs`

Action	Verb	What it does
`Index()`	GET	Renders `Views/Home/Index.cshtml` (marketing homepage)
`Privacy()`	GET	Static privacy page
`Contact()`	GET	Static contact page
`Error()`	GET	`[ResponseCache(NoStore=true)]` — renders `Error.cshtml` with `ErrorViewModel { RequestId }`

No services injected; just `ILogger<HomeController>` for diagnostics.

4.2 `StoreController.cs`

- Ctor: `IProductService`.
- `Index()` — loads all products, if `?q=...` filters in-memory on upper-cased `Name` / `Description`, sets `ViewBag.Q`, returns the list.

4.3 `BookingController.cs`

- Ctor: `IBookingService`.
- Actions:

Action	Verb	Auth	Flow
`Index()`	GET	Public	Empty `CreateBookingVm`
`Confirm(vm)`	POST `[ValidateAntiForgeryToken]`	Public	Validates VM; concatenates `deviceType: deviceModel`; delegates to `IBookingService`; on success redirects to `Success?ticket=...`; on failure re-renders `Index.cshtml` with errors
`Success(ticket)`	GET	Public	`ViewBag.Ticket`; shows the ticket UI
`Admin()`	GET	`[Authorize(Roles="Admin")]`	`GetAllAsync`; if `?q=` filters by ticket/name/phone (in-memory)
`UpdateStatus([FromBody] req)`	POST `[ValidateAntiForgeryToken]`	Admin	JSON body `{ BookingId: Guid, Status: BookingStatus }`; returns `{ status: "InProgress" }`
`Track()`	GET	Public	Empty `TrackBookingVm`
`Track(vm)`	POST `[ValidateAntiForgeryToken]`	Public	Calls `GetByTicketAsync`; sets `vm.Result`; if null, adds `ModelState` error

Authenticated bookings tag `UserId = User.Identity?.Name` (that's the email because `UserName = Email` in Identity).

4.4 `CartController.cs`

- Ctor: `ICartService` + `PixelBitDbContext` (direct DB access for `Orders`).
- Actions:

Action	Verb	Notes
`Index()`	GET	Renders current cart lines + total
`Add([FromBody] AddToCartRequest)`	POST `[ValidateAntiForgeryToken]`	JSON `{ productId, quantity }`; product existence checked; returns `{count,total}` for the AJAX badge
`Clear()`	POST	Clears session cart, redirects to `Index`
`Confirm()`	POST	Creates `Order` + `OrderItem[]` (FK patched after assignment), clears cart, redirects to `Success/{id}`
`Success(Guid id)`	GET	Loads order (`Include Items → Product`) for display

```
record AddToCartRequest(int ProductId, int Quantity);
```

4.5 `ProfileController.cs` — `[Authorize]` class-level

Action	Returns
`Index()`	profile landing page
`MyOrders()`	Orders where `UserId == User.Identity?.Name`, ordered DESC
`MyRepairs()`	Bookings where `UserId == User.Identity?.Name`, ordered DESC

4.6 `AdminController.cs` — `[Authorize(Roles="Admin")]`

Ctor: `PixelBitDbContext`, `UserManager<IdentityUser>`.

Action	Verb	What it does
`Index()`	GET	Builds `AdminDashboardVm`: counts, revenue summed over completed bookings/orders (SQLite `SumAsync<double?>` cast to decimal), top 8 device models grouped by count
`Users()`	GET	`_db.Users.OrderBy(Email).ToListAsync()`
`DeleteUser(id)`	POST `[ValidateAntiForgeryToken]`	Refuses to delete current admin; `UserManager.DeleteAsync`; toast on error
`Orders()`	GET	All orders with `Items → Product`, DESC by date
`OrderDetails(Guid id)`	GET	Single order with items
`UpdateOrderStatus(id, status)`	POST `[ValidateAntiForgeryToken]`	Mutates `Order.Status`
`Products()`	GET	All products ordered by name
`ProductCreate()`	GET	Empty `AdminProductEditVm`
`ProductCreate(vm)`	POST `[ValidateAntiForgeryToken]`	Adds product (trimmed), redirects to `Products`
`ProductEdit(int id)`	GET	Loads product into `AdminProductEditVm`
`ProductEdit(id, vm)`	POST `[ValidateAntiForgeryToken]`	Mutates tracked entity + saves
`ProductDelete(id)`	POST `[ValidateAntiForgeryToken]`	Removes product

4.7 `AdminUsersApiController.cs`

```
[ApiController]
[Route("api/admin")]
[Authorize(Roles = "Admin")]
public class AdminUsersApiController : ControllerBase
{
    [HttpGet("users")]
    public async Task<ActionResult<IReadOnlyList<AdminUserRowDto>>> GetUsers(CancellationToken ct) {
... }
}
public record AdminUserRowDto(
    string Id, string? Email, string? UserName,
    bool EmailConfirmed, string? PhoneNumber,
    DateTimeOffset? LockoutEnd, bool TwoFactorEnabled);
```

Used by the `Admin/Users` view's client-side table fetch. Unauthenticated API callers get **401** (not a redirect) because of the cookie `OnRedirectToLogin` override.

4.8 `LanguageController.cs`

```
[HttpPost, ValidateAntiForgeryToken]
public IActionResult Set(string culture, string returnUrl)
{
    culture = (culture ?? "en").ToLowerInvariant();
    if (culture is not ("en" or "ar")) culture = "en";
    Response.Cookies.Append(
        CookieRequestCultureProvider.DefaultCookieName,
        CookieRequestCultureProvider.MakeCookieValue(new RequestCulture(culture)),
        new CookieOptions { IsEssential = true, HttpOnly = false, Expires = 1 year });
    if (!Url.IsLocalUrl(returnUrl)) returnUrl = Url.Content("~/");
    return LocalRedirect(returnUrl);
}
```

The navbar's EN/AR toggle posts here with the current path as `returnUrl`.

---

5. Web — ViewModels (`PixelAndBit.Web/Models`)

`CreateBookingVm.cs`

- `[Required] CustomerName` (2–150)
- `[Required] PhoneNumber` — regex `^07\d{8}$`
- `[Required] DeviceType` (default `"Phone"`)
- `[Required] DeviceModel` (2–200)
- `[Required] IssueDescription` (20–2000)
- `[Range(0, 999999)] decimal? EstimatedCost`

Display names and error messages are **localizer keys** resolved via `SharedResource.resx`.

`TrackBookingVm.cs`

- `[Required] TicketReference` — regex `^PB-\d{4}-[A-Z0-9]{4}$`
- `Booking? Result` populated after a successful lookup

`AdminDashboardVm.cs`

- Totals: `TotalProducts`, `PendingRepairs`, `TotalRequests`, `TotalUsers`
- Revenues: `ServicesRevenueJod`, `SalesRevenueJod`
- `IReadOnlyList<(string DeviceModel, int Count)> TopDeviceModels`

`AdminProductEditVm.cs`

- `int Id`
- `[Required, StringLength(120)] string Name`
- `[Range(0, 1_000_000)] decimal Price`
- `[Required, StringLength(2000)] string Description`
- `[Range(0, 1_000_000)] int StockQuantity`
- `[StringLength(500)] string? ImageUrl`

`ErrorViewModel.cs`

```
public string? RequestId { get; set; }
public bool ShowRequestId => !string.IsNullOrEmpty(RequestId);
```

---

6. Web — Razor Pages (Identity area)

All pages live under `PixelAndBit.Web/Areas/Identity/Pages/Account/`. `_ViewStart.cshtml` sets `Layout = "/Views/Shared/_Layout.cshtml"`, so the auth pages render under the same navbar/theme as the rest of the site.

6.1 `Login.cshtml(.cs)`

- Inputs: `Email`, `Password`, `RememberMe`.
- Shows a **"Forgot password?"** link and — when `AllowPublicRegistration` is true — a **"Create account"** button under the Login button.
- Renders `pb_toast` / `pb_toast_error` (used by the reset flow on return).
- `OnPostAsync`:

1. Validates.
2. If the account exists but email is **not** confirmed, redirects to `VerifyEmail` page.
3. Otherwise calls `SignInManager.PasswordSignInAsync(email, password, remember, lockoutOnFailure: false)`.
4. On success: `LocalRedirect(returnUrl ?? "~/")`.
5. On failure: generic `"Invalid login attempt."`.

6.2 `Register.cshtml(.cs)`

- Gated by `_configuration.GetValue("AllowPublicRegistration", false)`.
- Creates user with `EmailConfirmed = false`, generates a 5-digit code, hashes it, stores in `EmailVerificationCodes` (10-min TTL, 6 attempts), sends email with `VerifyEmailModel.BuildEmailHtml(code, includeRegistrationWelcome: true)`.
- Redirects to `/Identity/Account/VerifyEmail?email=...`.

6.3 `VerifyEmail.cshtml(.cs)`

The richest page.

- Ctor DI: `PixelBitDbContext`, `UserManager<IdentityUser>`, `SignInManager<IdentityUser>`, `IEmailSender`.
- Constants: `ResendCooldownPeriodSeconds = 60`, `MaxResendsPerHour = 10`.
- `OnGet(email, returnUrl)` — bails to `Register` if no email; reads `ResendCooldownSeconds` from the latest code timestamp.
- `OnPostAsync`:

1. Find user by email; if already confirmed → sign in and redirect.
2. Get latest **non-consumed, non-expired** code for this user/email.
3. If none → "Code expired".
4. If `Attempts >= MaxAttempts` → "Too many attempts".

5. Increment attempts; `FixedTimeEquals(HashCode(userId, Email, Input.Code), rec.CodeHash)`.
6. On match: stamp `ConsumedAtUtc`, set `EmailConfirmed = true`, `SignInAsync`, redirect.

- `OnPostResendAsync(email, returnUrl)`:
- Enforces 60s cooldown on *last* issued code.
- Enforces ≤ 10 codes / hour per user.
- Generates + stores + emails a new code.
- Static helpers reused across the app:
- `Generate5DigitCode()` — `RandomNumberGenerator.GetInt32(10000,100000)`
- `HashCode(userId, email, code) = SHA256(userId|EMAIL_UPPER|code)` hex
- `FixedTimeEquals(a,b)` — byte-length guard + `CryptographicOperations.FixedTimeEquals`
- `BuildEmailHtml(code, includeRegistrationWelcome)` — light, transactional, mobile-friendly HTML (see `03_Frontend.md`).

6.4 `Logout.cshtml(.cs)`

- `OnPost` signs out and redirects to `~/`.
- `OnGet` redirects back to login — prevents accidental GET-based logout from CSRF.

6.5 `AccessDenied.cshtml(.cs)`

Minimal: sets `ReturnUrl` from query; view offers **Home** / **Sign in** links with the glassy theme.

6.6 `RegisterConfirmation.cshtml(.cs)`

Plain confirmation placeholder (not actively used in the post-register flow — we go straight to `VerifyEmail`).

6.7 Forgot-password trio (session-backed)

`PasswordResetFlow.cs`

A static helper, **no DB writes**:

- Session key constants: `K_Email`, `K_UserId`, `K_CodeHash`, `K_ExpiresTicks`, `K_Attempts`, `K_MaxAttempts`, `K_ResetToken`, `K_Verified`, `K_LastSentTicks`.
- Config: `CodeTtlMinutes = 15`, `ResendCooldownSeconds = 60`, `DefaultMaxAttempts = 6`.
- Helpers:
- `HasPendingEmail(session)` — gate for Step 2 access.
- `HasPendingCode(session)` — only true when a real code was issued.
- `IsCodeVerified(session)` — gate for Step 3 access (verified + token + userId).
- `ResendCooldownSeconds_(session)` — reads `K_LastSentTicks`.
- `ClearAll(session)` — wipes all keys.
- `Generate5DigitCode()` / `HashCode("reset|userId|EMAIL|code")` / `FixedTimeEquals(...)`
- `BuildResetEmailHtml(code)` — same light Amazon-style HTML as verify.

`ForgotPassword.cshtml(.cs)` (Step 1)

- Inputs: `Email`.

- `OnPostAsync`:

1. `ClearAll(session)`.
2. **Always** store `email`, `attempts=0`, `maxAttempts`, `lastSentTicks` in session (so Step 2 is reachable regardless of existence).
3. If the user exists AND has confirmed email:

- Generate code, call `UserManager.GeneratePasswordResetTokenAsync(user)` (Identity's real token), store code hash + userId + token + 15-min expiry in session, `SendAsync` the email.

1. Else: small delay to blur timing; no session token/hash stored.
2. Redirect to Step 2 + toast: "*If an account exists…*".

`VerifyResetCode.cshtml(.cs)` (Step 2)

- `OnGet` — redirects to Step 1 if `!HasPendingEmail`.
- `OnPost`:

1. Gate on `HasPendingEmail`.
2. Read `userId/email/codeHash/expTicks/attempts/max`.
3. If `attempts >= max` → wipe + redirect to Step 1 with toast.
4. Increment `attempts` **before** comparing (bounded brute-force).
5. If no `codeHash/userId` (email had no real account) → same generic `"Wrong code. Please try again."` response.
6. If expired → wipe + redirect to Step 1.
7. `FixedTimeEquals(HashCode(userId,email,code), codeHash)` — mismatch keeps user on Step 2.
8. Success: set `K_Verified = "1"`, remove `K_CodeHash` (one-time use), redirect to Step 3.

- `OnPostResendAsync` — always advances cooldown marker (timing parity), re-resolves the user, re-issues code if a real user exists; always toast "*If an account exists…*".

`ResetPassword.cshtml(.cs)` (Step 3)

- `OnGet` / `OnPost` both gate on `IsCodeVerified`. Without it, user is redirected to Step 1 with error toast.
- `OnPost`:

1. Load `user` by `K_UserId` and `token` by `K_ResetToken`.
2. `UserManager.ResetPasswordAsync(user, token, Input.Password)`.
3. `UserManager.UpdateSecurityStampAsync(user)` (invalidates existing sign-ins).
4. `ClearAll(session)`.
5. Redirect to Login with success toast.

Continue to `03_Frontend.md` for views, layouts, CSS system, and JS.

03 — Frontend: Views, Layout, Styling, JS, Localization

This file documents the UI layer: Razor views, the shared layout and navbar, the design system in `pixelbit.css`, `site.js`, and the localization/RTL pipeline.

---

1. Razor infrastructure

`Views/_ViewImports.cshtml`

```
@using PixelAndBit.Web
@using PixelAndBit.Web.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Enables `asp-*` tag helpers and imports your namespaces globally.

`Views/_ViewStart.cshtml`

```
@{
    Layout = "_Layout";
}
```

Applies `_Layout.cshtml` to every MVC view. The Razor Pages Identity pages have their own `_ViewStart` under `Areas/Identity/Pages/` that **also** points to `/Views/Shared/_Layout.cshtml` — guaranteeing a single consistent layout for the entire site.

`Views/Shared/_Layout.cshtml`

- Sets `<html lang="@lang" dir="@(isAr ? "rtl" : "ltr")">`.
- Preconnects Google Fonts and loads 4 families: **Inter** (body), **Space Grotesk** (headings), **Orbitron** (brand/ticket), **Cairo** (Arabic heading).
- Picks the Bootstrap bundle:

```
@if (isAr)  <link href="~/lib/bootstrap/.../bootstrap.rtl.min.css" />
@else       <link href="~/lib/bootstrap/.../bootstrap.min.css" />
```

- Loads `site.css`, `pixelbit.css`, and the per-project scoped CSS.
- `<body class="@(isAr ? "pb-rtl" : "pb-ltr") pb-has-fixed-header">`.
- Fixed header wraps `<partial name="_Navbar">`.
- `<main id="main-content">` + content container with `pb-section pb-fade`.
- Footer: brand, Google Maps link, WhatsApp link, Instagram link — all localized.
- Injects a CSRF token to JS:

```
  <script>
      window.__pb = window.__pb || {};
      window.__pb.csrf =
@Html.Raw(JsonSerializer.Serialize(Antiforgery.GetAndStoreTokens(Context).RequestToken));
  </script>
```

`site.js` and any other AJAX send this token back via the `RequestVerificationToken` header — that's how `[ValidateAntiForgeryToken]` on APIs like `Cart/Add` is satisfied without `<form>` POSTs.

`Views/Shared/_Navbar.cshtml`

- Active-link tracking: inspects `ViewContext.RouteData.Values["controller"/"action"]`.
- **Live cart badge**: reads `await CartService.GetItemCountAsync()` on every render; wrapped in a `pb-cart-badge` element with id `pb-cart-badge` updated via JS after Add/Remove.
- **Language toggle** (`<form method="post" asp-controller="Language">`), separate compact version for mobile top-right cluster.
- **Role-aware admin link**:

```
  @if (User.IsInRole("Admin")) { ...Admin link... }
```

- **Auth actions**:
- When signed in → "Profile" link + logout form (POST, CSRF-safe).
- When anonymous → "Sign in" link to `~/Identity/Account/Login`.
- **Mobile drawer** (`offcanvas`): mirrors all nav links + auth actions with the same role checks.
- Mobile hamburger button triggers `#pbMobileNav`; cart button also shown on small screens so users always see their cart.

`Views/Shared/Error.cshtml`

Minimal error view; renders `RequestId` when present.

`Views/Shared/_ValidationScriptsPartial.cshtml`

Includes jquery-validation + jquery-validation-unobtrusive so `asp-validation-for` / `asp-validation-summary` show real-time client validation.

---

2. Page views (MVC)

2.1 `Home/Index.cshtml`

Marketing homepage. Key sections (each wrapped in `.pb-saas-section`):

- **Hero** (`.pb-hero2`) — aurora, grid, sparks, CTA buttons ("Book repair" / "Track ticket" / anchor link).
- **Benefits strip** (`.pb-benefits-strip`) — 3 icon cards (Precise Diagnosis / Quality Repair / Quick Service).
- **Value** — 3 cards (Fast booking / Transparent process / Track anytime).

- **Features** — 2 tall cards (included-with-service tag grid + Store CTA).
- **Services** — 3 service cards linking to Booking / Store / Track.
- **Work** — Instagram-linked gallery thumbnails.
- **Actions** (*Get Started Instantly*) — 3 equal-height cards with identical full-width CTAs (`.pb-action-btn`).
- **Visit** — full-bleed gradient section with a Google Maps CTA.

All grid columns use `col-12 col-md-6 col-lg-4 d-flex` + `w-100 d-flex flex-column` pattern so cards stretch equal-height and CTAs sit at the bottom via `mt-auto` .

2.2 `Home/Contact.cshtml` , `Home/Privacy.cshtml`

Static informational pages.

2.3 `Store/Index.cshtml`

- Store brand header, search form (`<form method="get">?q=`).
- Product grid with **card flex-column** and CTA at bottom.
- **Add-to-cart** is AJAX (`POST /Cart/Add` with `RequestVerificationToken` header); response updates `#pb-cart-count` .

2.4 `Booking/Index.cshtml`

- Bound to `CreateBookingVm` .
- **Device chips** (`.pb-chip`) — JS toggles `is-active` and writes the value into a hidden `<input asp-for="DeviceType">` .
- Phone input uses `pb-ltr-inline` so numbers stay LTR inside an Arabic RTL page.
- Submit button: full-width `.pb-btn-glow` on the primary gradient.

2.5 `Booking/Track.cshtml`

- `TrackBookingVm` + small lookup form.
- After lookup, renders the ticket reference (Orbitron font) and a status `badge` .

2.6 `Booking/Success.cshtml`

- Displays `ViewBag.Ticket` prominently with copy-to-clipboard friendly formatting.

2.7 `Booking/Admin.cshtml`

- Admin-only list of bookings with search, status dropdown per row. Each dropdown change fires a fetch to `Booking/UpdateStatus` with the CSRF header.

2.8 `Cart/Index.cshtml`

- List of `CartLine` (product + qty + subtotal).
- "Clear cart" POST form; "Confirm" POST form; both CSRF-protected.
- Shows `ViewBag.Total` with thousands separator.

2.9 `Cart/Success.cshtml`

- Order detail view. Items listed with unit price and line totals.

2.10 `Profile/Index.cshtml`, `Profile/MyOrders.cshtml`, `Profile/MyRepairs.cshtml`

- Landing + two list views for the signed-in user's history.

2.11 Admin views

View	Purpose
`Admin/Index.cshtml`	Dashboard cards (counts, revenue, top-8 device models with progress bars) + quick-action buttons
`Admin/Users.cshtml`	Users table — populated client-side via `GET /api/admin/users` (JSON)
`Admin/Products.cshtml`	Products list + CRUD links
`Admin/ProductCreate.cshtml` / `ProductEdit.cshtml`	Bound to `AdminProductEditVm`
`Admin/Orders.cshtml`	All orders list
`Admin/OrderDetails.cshtml`	Single order + line items + status update

---

3. Identity views (Razor Pages)

All under `Areas/Identity/Pages/Account/*.cshtml`. They share:

- `.pb-auth-shell` — centered, max-width, responsive container
- `.pb-auth-card` — the dark glassy card with blurred backdrop
- `.pb-auth-btn` — 46 px min-height, 14 px radius, consistent typography
- `.pb-auth-link` — muted text-like link style
- `.pb-auth-divider` — `"New here?"` style rule with `<span>` label

`Login.cshtml`

- Email + password + "Remember me".
- "Forgot password?" link next to remember-me checkbox.
- "Create account" button (gated by `Model.AllowPublicRegistration`).
- Renders `TempData["pb_toast"/"pb_toast_error"]` (used by reset flow).

`Register.cshtml`

Simple email + password + confirm. Submits to create user and triggers the verify flow.

`VerifyEmail.cshtml`

- Masked email banner + centered 5-digit input (`inputmode="numeric"`, `autocomplete="one-time-code"`).
- JS strips non-digits and auto-submits at 5 digits.

- Resend button with countdown hint (`ResendCooldownSeconds` initial).
- Displays `pb_toast` / `pb_toast_error` in coloured banner blocks.

`ForgotPassword.cshtml`

Email input + "Send reset code" + "Back to sign in".

`VerifyResetCode.cshtml`

Same 5-digit input pattern as VerifyEmail, plus resend form.

`ResetPassword.cshtml`

New password + confirm + submit. Guarded server-side by `IsCodeVerified(session)`.

`Logout.cshtml` / `AccessDenied.cshtml`

Minimal. Logout accepts POST only (CSRF safety).

---

4. Design system — `wwwroot/css/pixelbit.css`

The stylesheet is ~2,800 lines and implements the entire visual language. Key conceptual groups:

4.1 Theme primitives / variables

- `:root` — CSS custom properties for colors, durations, easings (e.g. `--pb-dur-3`, `--pb-ease`).
- Font stacks: `Inter`, `Space Grotesk`, `Orbitron`, `Cairo`.

4.2 Buttons & gradients

- **`.pb-btn-glow`** — a hoverable glow around Bootstrap `btn` variants.
- **`.pb-nav-cta-gradient`** — the pink→purple→cyan gradient CTA pill (used for "Book repair" in navbar).
- Primary button gradient is defined with 3 stops (cyan → purple → blue).

4.3 Surfaces

- **`.pb-surface`** + **`.pb-card`** — glassmorphism surface (`background rgba(255,255,255,.04)` + `backdrop-filter: blur(14-18px)` + inner border shadow).
- **`.pb-saas-card`**, **`.pb-service2`**, **`.pb-benefit`** — layered card families used in the homepage.
- **`.pb-action-card`** — uses `display:flex; flex-direction:column; padding:1.4rem 1.3rem;` so `.pb-action-btn` can `margin-top:auto`.
- **`.pb-action-btn`** — 46 px min-height, 14 px radius, `inline-flex center center`, `white-space:nowrap` — the rule that keeps the three "Get Started Instantly" buttons identical.

4.4 Hero

- **`.pb-hero2`** with layered backgrounds: `.pb-hero2-bg` (radial), `.pb-hero2-aurora`, `.pb-hero2-grid`, `.pb-hero2-sparks`.

- Parallax-friendly: elements with `data-parallax` are wiggled by `site.js`.

4.5 Navbar

- `.pb-navbar-next` has a soft glassy surface, turns `--scrolled` when the page scrolls past 8 px.
- `.pb-nav-pill-wrap` — the rounded nav-pill container on desktop.
- `.pb-nav-icon-btn` — 40×40 icon buttons (cart, hamburger) with focus rings.
- `.pb-offcanvas` — mobile drawer styling (dark blur, brand header, same gradient CTAs).

4.6 Auth shell (added in recent iterations)

```
.pb-auth-shell  { max-width: 28rem; margin: 0 auto; padding-inline: .25rem; }
.pb-auth-card   { backdrop-filter: blur(15px) saturate(1.08); }
.pb-auth-btn    { width:100%; min-height:46px; border-radius:14px; font-weight:600; letter-
spacing:.01em; }
.pb-auth-link   { text-decoration:none; color: rgba(235,235,235,.78); }
.pb-auth-divider{ text-align:center; letter-spacing:.18em; text-transform:uppercase; ... }
```

4.7 Responsiveness

- **Breakpoints** used with Bootstrap: `sm(576)`, `md(768)`, `lg(992)`, `xl(1200)`.
- Custom media queries for nav, hero, cards, section rhythm at `max-width: 991.98`, `767.98`, `575.98`, `430`, `360`.
- Mobile hardening block near the bottom of the file:

```
html, body { overflow-x: hidden; }
@media (max-width: 575.98px) {
    .pb-nav-surface { gap:.5rem; flex-wrap:nowrap; }
    .pb-navbar-brand-next { min-width: 0; }
    .pb-nav-icon-btn { min-width: 40px; min-height: 40px; }
}
```

4.8 RTL

- `.pb-rtl` overrides flip text-alignment where needed and select Cairo font for Arabic headings.
- `.pb-ltr-inline` forces phone numbers / ticket codes / times to render LTR inside RTL paragraphs.

4.9 Motion

- `.pb-fade` / `.pb-anim` get a `pb-in` class added by `site.js` when scrolled into view, driving a fade/rise/tilt/glow/pop/sheen entrance — controlled by:

```
@media (prefers-reduced-motion: reduce) {
    .pb-fade, .pb-anim, .pb-saas-card, .pb-service2, ... { transition: none !important; }
}
```

4.10 Other component groups

- `.pb-work2` (Instagram gallery tiles with photo, chip, overlay)

- `.pb-visit` (full-bleed final CTA)
- `.pb-saas-cta` (generic section CTA band)
- `.pb-lang-toggle` / `.pb-lang-btn` (EN/AR pill)

---

5. `wwwroot/js/site.js`

A single, framework-free module (immediately invoked). Responsibilities:

1. **Fade-in on scroll** — `IntersectionObserver` adds `pb-in` to `.pb-fade` / `.pb-anim` elements; supports `data-pb-stagger="1..12"` for staggered reveals. Gracefully downgrades to unconditional reveal on older browsers.
2. **Scroll parallax variable** — sets `--pb-scroll` on `<html>` on `scroll` via `requestAnimationFrame`.
3. **Navbar scroll state** — toggles `.pb-navbar--scrolled` after 8 px.
4. **Device chips (booking form)** — listens on `.pb-chip` clicks, updates the hidden `DeviceType` input and toggles `is-active`.
5. **Cart AJAX** — all `form[data-cart-add]` or store card buttons call `fetch('/Cart/Add', { method:'POST', headers:{RequestVerificationToken:__pb.csrf}, body: JSON.stringify({productId, quantity}) })` and update `#pb-cart-count`.
6. **Keyboard safety** — closes offcanvas on Esc; focus-visible polyfills; prevents scroll jank on anchor links.

The individual auth pages include small inline scripts (see `VerifyEmail.cshtml` / `VerifyResetCode.cshtml`) for the 5-digit input auto-submit and the resend countdown.

---

6. Localization + RTL

6.1 Config (`Program.cs`)

```
builder.Services.AddLocalization();
builder.Services.AddControllersWithViews()
    .AddViewLocalization()
    .AddDataAnnotationsLocalization(o =>
        o.DataAnnotationLocalizerProvider = (_, f) => f.Create(typeof(SharedResource)));

var supportedCultures = new[] { "en", "ar" }.Select(c => new CultureInfo(c)).ToList();
app.UseRequestLocalization(new RequestLocalizationOptions
{
    DefaultRequestCulture = new RequestCulture("en"),
    SupportedCultures = supportedCultures,
    SupportedUICultures = supportedCultures,
    RequestCultureProviders = new[] { new CookieRequestCultureProvider() }
});
```

Only cookie-based culture resolution is used (so language selection persists, it's not locked to browser language).

6.2 Resource files

- `Resources/SharedResource.cs` — marker class for the resource lookup key.
- `Resources/SharedResource.resx` — English strings.

- `Resources/SharedResource.ar.resx` — Arabic strings.

6.3 Usage in views

```
@inject Microsoft.Extensions.Localization.IStringLocalizer<PixelAndBit.Web.SharedResource> T
<h1>@T["Home.Hero.Title"]</h1>
<span>@T["Store.ItemsCount", list.Count, suffix]</span>
```

6.4 ViewModels

ViewModels use the same keys via `Display(Name = "...")` + `ErrorMessage = "Validation.Required"`, and localization is wired through `DataAnnotationLocalizerProvider` → `SharedResource`.

6.5 Switching language

The navbar contains two `<form method="post" asp-controller="Language" asp-action="Set">` sub-buttons, passing `culture=en|ar` + current `returnUrl`. `LanguageController` writes the `.AspNetCore.Culture` cookie with 1-year expiry and `LocalRedirect`s back.

6.6 RTL presentation

- `_Layout.cshtml` writes `<html dir="rtl">` when culture is `ar` and loads `bootstrap.rtl.min.css` instead of the LTR bundle.
- `body` gets `pb-rtl` class, targeted by `pixelbit.css` rules to flip text-align, swap margins, and apply Cairo font to headings.
- `.pb-ltr-inline` is a utility we apply to phone numbers, ticket codes, time ranges, and WhatsApp numbers so digits never reverse.

---

7. CSRF + cookies in the UI

- Every form uses `@Html.AntiForgeryToken()` or the `asp-*` helpers that emit the hidden `__RequestVerificationToken` input.
- For JSON POSTs (cart add, booking admin status update, admin users API), the token is exposed via `window.__pb.csrf` in `_Layout.cshtml` and passed back as the `RequestVerificationToken` HTTP header.
- Cookies:
- `.AspNetCore.Identity.Application` — auth cookie (Identity default).
- `.PixelAndBit.Session` — session for cart + password-reset state.
- `.AspNetCore.Culture` — language preference.
- `.AspNetCore.Antiforgery.*` — CSRF token binding.

Continue to `04_Database_Auth_Config.md` for the schema, authorization rules, and configuration files.

04 — Database, Authentication & Authorization, Configuration

This file covers the data layer, identity/security, and configuration.

---

1. Database: `PixelBitDbContext`

Defined in `PixelAndBit.Infrastructure/Data/AppDbContext.cs`:

```
public class PixelBitDbContext : IdentityDbContext<IdentityUser>
{
    public DbSet<Product>                Products               => Set<Product>();
    public DbSet<Booking>                Bookings               => Set<Booking>();
    public DbSet<Order>                  Orders                 => Set<Order>();
    public DbSet<OrderItem>              OrderItems             => Set<OrderItem>();
    public DbSet<EmailVerificationCode>  EmailVerificationCodes => Set<EmailVerificationCode>();
    public DbSet<RepairService>          RepairServices         => Set<RepairService>();
    public DbSet<Appointment>            Appointments           => Set<Appointment>();

    protected override void OnModelCreating(ModelBuilder b)
    {
        base.OnModelCreating(b);                                  // Identity schema
        b.ApplyConfigurationsFromAssembly(typeof(PixelBitDbContext).Assembly);
    }
}
```

- Inherits from `IdentityDbContext<IdentityUser>` which brings the standard ASP.NET Identity 7-table schema.
- `ApplyConfigurationsFromAssembly` auto-registers every `IEntityTypeConfiguration<T>` in `Infrastructure.Data.Configurations/`.

1.1 Provider & connection

`Program.cs`:

```
var sqlitePath = Path.Combine(builder.Environment.ContentRootPath, "pixelbit.db");
builder.Configuration.AddInMemoryCollection(new Dictionary<string, string?>
{
    ["ConnectionStrings:PixelBitConnection"] = $"Data Source={sqlitePath}"
});

builder.Services.AddDbContext<PixelBitDbContext>(opts =>
    opts.UseSqlite(sqliteConnectionString,
        sql => sql.MigrationsAssembly("PixelAndBit.Infrastructure")));
```

This makes the database file always live next to the app (beside `Program.cs` in dev; beside the executable in production). The connection string in `appsettings.json` is overridden at startup.

1.2 Initialization path

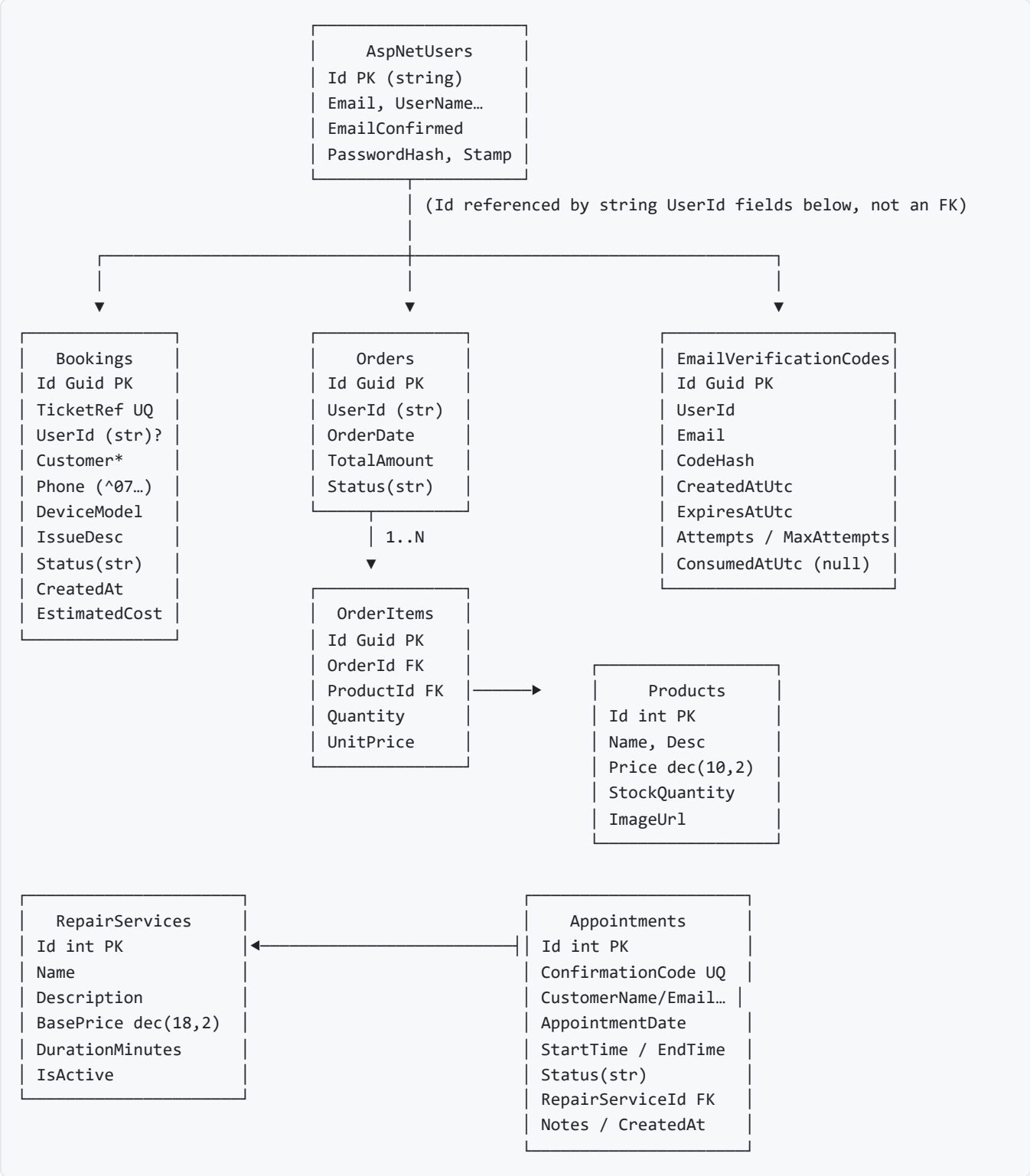
- `EnsureDatabaseInitializedAsync()` runs on first request via a minimal `app.Use(...)` middleware (see `Program.cs`).
- Calls `await db.Database.MigrateAsync()` — applies any pending migrations automatically.
- In **Development** only, calls `DbSeeder.SeedAsync(...)` after migration.

1.3 Migrations

Under `PixelAndBit.Infrastructure/Migrations/`:

- `20260418124033_InitialSqlite.cs` — creates everything:
- **Identity tables**: `AspNetRoles`, `AspNetUsers`, `AspNetRoleClaims`, `AspNetUserClaims`, `AspNetUserLogins`, `AspNetUserRoles`, `AspNetUserTokens`.
- **Business tables**: see table below.
- Indexes: unique `Bookings.TicketReference`, unique `Appointments.ConfirmationCode`, composite index on `EmailVerificationCodes(UserId, Email, ConsumedAtUtc, ExpiresAtUtc)`.
- `PixelBitDbContextModelSnapshot.cs` — EF model snapshot.

1.4 Schema summary



1.5 Per-table highlights (from Fluent configs)

Table	Key	Important constraints
`Products`	`Id` int identity	`Name` required ≤200, `Description` required ≤2000, `Price decimal(10,2)`, `ImageUrl` ≤500
`Bookings`	`Id` Guid	`TicketReference` unique ≤20, `UserId` ≤450 nullable, `Phone` ≤20, `DeviceModel` ≤200, `IssueDescription` ≤2000, `Status` stored as string ≤30, `EstimatedCost decimal(10,2)`
`Orders`	`Id` Guid	`UserId` ≤450 nullable, `TotalAmount decimal(10,2)`, `Status` stored as string ≤20, `HasMany(Items).OnDelete(Cascade)`
`OrderItems`	`Id` Guid	`UnitPrice decimal(10,2)`, `HasOne(Product).OnDelete(Restrict)` → prevents deleting products that are part of an order
`Appointments`	`Id` int	`ConfirmationCode` unique ≤20, `Status` stored as string ≤20, `HasOne(RepairService).OnDelete(Restrict)`
`RepairServices`	`Id` int	`BasePrice decimal(18,2)`
`EmailVerificationCodes`	`Id` Guid	Index on `(UserId, Email, ConsumedAtUtc, ExpiresAtUtc)`

1.6 Relationships at a glance

- `Order 1..N OrderItem` — cascade delete from `Order`. Items removed with the order.
- `OrderItem N..1 Product` — restrict delete. You can't delete a product if it's in an order.
- `Appointment N..1 RepairService` — restrict delete.
- `IdentityUser → Bookings / Orders / EmailVerificationCodes` — *no enforced FK*. Stored as `string UserId` only. This allows guest bookings/orders (null `UserId`) and avoids EF navigating back to the Identity tables by accident.
- No explicit user table for application-level profiles — we rely on Identity's `AspNetUsers`.

1.7 Where raw data comes from

- **Seed** (`DbSeeder.cs`, Development only):
- Roles `Admin`, `Customer`.
- Admin user `admin@pixelbit.jo` / `Admin@Pb2026!` (`EmailConfirmed=true`).
- 5 seed products (thermal paste / keyboard modding / console cleaning / HDMI mod / entry-level build).
- **Runtime** — everything else is user-generated via the web UI / APIs.

1.8 Health endpoint

`GET /__health/db` returns `{ ok: true, products: <count>, database: "sqlite", file: "<path>" }` — useful for load balancer probes.

---

2. Authentication

2.1 Configuration (`Program.cs`)

```
builder.Services.AddIdentity<IdentityUser, IdentityRole>(o =>
{
    o.Password.RequireDigit            = false;
    o.Password.RequiredLength          = 6;
    o.Password.RequireNonAlphanumeric  = false;
    o.Password.RequireUppercase        = false;
    o.SignIn.RequireConfirmedAccount   = true;  // blocks login until email is verified
})
.AddEntityFrameworkStores<PixelBitDbContext>()
.AddDefaultTokenProviders()
.AddDefaultUI();

builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath        = "/Identity/Account/Login";
    options.AccessDeniedPath = "/Identity/Account/AccessDenied";
    options.Events = new CookieAuthenticationEvents
    {
        OnRedirectToLogin = ctx =>
        {
            if (ctx.Request.Path.StartsWithSegments("/api"))
            {
                ctx.Response.StatusCode = 401;      // APIs → proper status code
                return Task.CompletedTask;
            }
            ctx.Response.Redirect(ctx.RedirectUri);
            return Task.CompletedTask;
        },
        OnRedirectToAccessDenied = ctx =>
        {
            if (ctx.Request.Path.StartsWithSegments("/api"))
            {
                ctx.Response.StatusCode = 403;
                return Task.CompletedTask;
            }
            ctx.Response.Redirect(ctx.RedirectUri);
            return Task.CompletedTask;
        }
    };
});
```

- **Password policy** is intentionally relaxed (length 6, no uppercase/digit required) to reduce friction; raise these in production if needed.
- `SignIn.RequireConfirmedAccount = true` is what makes the custom **5-digit email verification** mandatory.
- Cookie events convert redirects to 401/403 for `/api/*` paths — critical for the admin users JSON API.

2.2 Registration + email verification (custom 5-digit flow)

Files: `Areas/Identity/Pages/Account/Register.cshtml.cs`, `VerifyEmail.cshtml.cs`.

1. `Register.OnPostAsync` creates the `IdentityUser` with `EmailConfirmed=false`.
2. Generates a 5-digit code, hashes it `SHA256(userId|EMAIL_UPPER|code)`, stores it in `EmailVerificationCodes` (expiry 10 min, 6 attempts).

3. Sends the code via `IEmailSender.SendAsync` using the clean light HTML from `VerifyEmailModel.BuildEmailHtml(code, includeRegistrationWelcome: true)`.
4. Redirects to `/Identity/Account/VerifyEmail?email=...`.
5. User enters code → compared in constant time → sets `user.EmailConfirmed = true` → `SignInAsync`.

Rate-limits on verification:

- **Resend cooldown**: 60 seconds since the last code.
- **Hourly cap**: 10 codes per user per hour.

2.3 Login

File: `Login.cshtml.cs`.

```
var user = await _userManager.FindByEmailAsync(Input.Email);
if (user != null && !await _userManager.IsEmailConfirmedAsync(user))
    return RedirectToPage("./VerifyEmail", new { email = Input.Email, returnUrl });

var result = await _signInManager.PasswordSignInAsync(
    Input.Email, Input.Password, Input.RememberMe, lockoutOnFailure: false);
```

Same form serves users and admins. Role-based authorization picks up once they sign in.

2.4 Forgot password (multi-step)

Files: `ForgotPassword.cshtml.cs`, `VerifyResetCode.cshtml.cs`, `ResetPassword.cshtml.cs`, `PasswordResetFlow.cs`.

- State lives in **server session** (no DB). Keys are prefixed `pb.reset.*`.
- Step 1 always stores the submitted email (even when no account exists) so Step 2 is always reachable. A real code + Identity reset token are only stored when the account is real and email-confirmed.
- Step 2 enforces:
- `HasPendingEmail` gate
- attempt counter increment **before** comparison
- `codeHash/userId` presence check → same generic "Wrong code" response when absent (no info leak)
- expiry check (15 min)
- `CryptographicOperations.FixedTimeEquals` for constant-time compare
- one-time use (code hash removed on success)
- Step 3 gated on `IsCodeVerified` (verified flag + token + userId). Uses Identity's own `UserManager.ResetPasswordAsync(user, token, newPw)` + `UpdateSecurityStampAsync` to invalidate existing sign-in cookies.

2.5 Logout

`Logout.cshtml.cs`:

- `OnPost` → `SignOutAsync` → redirect to `~/`.
- `OnGet` → redirect to login (prevents CSRF via GET).

Navbar renders both the desktop and mobile logout as real POST forms with antiforgery tokens.

2.6 Where is the admin link in the navbar?

`Views/Shared/_Navbar.cshtml`:

```
@if (User.IsInRole("Admin"))
{
    <li class="nav-item">
        <a asp-controller="Admin" asp-action="Index">@T["Nav.Admin"]</a>
    </li>
}
```

The Admin link is therefore hidden entirely from the public (including anonymous users). Admin acquires visibility of admin features only after signing in.

---

3. Authorization

3.1 Role-based (`Admin`)

Applied at controller class level where appropriate:

Target	Attribute
`AdminController`	`[Authorize(Roles = "Admin")]`
`AdminUsersApiController`	`[Authorize(Roles = "Admin")]` + `[ApiController]` + `[Route("api/admin")]`
`BookingController.Admin`, `BookingController.UpdateStatus`	`[Authorize(Roles = "Admin")]` on those specific actions

3.2 Authenticated-only

- `ProfileController` — `[Authorize]` at class level (any signed-in user).

3.3 Anti-forgery (CSRF)

- Applied to every mutating POST via `[ValidateAntiForgeryToken]` or implicit binding via `asp-for` forms.
- JS-initiated POSTs pass `window.__pb.csrf` in the `RequestVerificationToken` header (see `_Layout.cshtml` + `site.js`).

---

4. Configuration

4.1 `appsettings.json`

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "AllowPublicRegistration": true,
  "ConnectionStrings": {
    "PixelBitConnection": "Data Source=pixelbit.db"
  },
  "Smtp": {
    "Host": "smtp.gmail.com",
    "Port": 465,
    "EnableSsl": true,
    "Username": "pixelbitjo@gmail.com",
    "Password": "mzfpqidcqozmugdx",      // Gmail app password
    "FromEmail": "pixelbitjo@gmail.com",
    "FromName": "Pixel & Bit"
  }
}
```

Important keys:

- `AllowPublicRegistration` — when false, `/Identity/Account/Register` redirects back to login with a toast.
- `ConnectionStrings:PixelBitConnection` — overwritten by `Program.cs` at boot with the absolute SQLite path.
- `Smtp` — bound to `SmtpEmailOptions`; if not properly configured (placeholders like `YOUR_HOST`), `NullEmailSender` is registered instead.

> ⚠️ **Secret**: the current `Smtp.Password` is a Gmail app password checked into source. In production, move this to environment variables / user secrets / a key vault.

4.2 `appsettings.Development.json`

```
{
  "AllowPublicRegistration": true,
  "Logging": { "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" } },
  "ConnectionStrings": { "PixelBitConnection": "Data Source=pixelbit.db" }
}
```

4.3 `appsettings.Production.json`

Same as base, with `LogLevel.Default: Warning` and the same `AllowPublicRegistration: true`.

4.4 Environment variables of interest

- `ASPNETCORE_ENVIRONMENT` = `Development` | `Production`
- `ASPNETCORE_URLS` = e.g. `http://127.0.0.1:5001` (overrides `Program.cs` defaults)

---

5. SMTP delivery

Implementation: `PixelAndBit.Infrastructure/Email/SmtpEmailSender.cs` using **MailKit / MimeKit**.

- Defaults: `smtp.gmail.com:465` with implicit SSL.
- Uses app-password authentication (Gmail).
- `BuildResetEmailHtml` / `BuildEmailHtml` produce a light, table-based, inline-styled, mobile-friendly HTML that renders reliably in Gmail/Outlook/Apple Mail.
- Logs to stdout for debugging (`=== SMTP DEBUG START / END / ERROR ===`).

Callers handle failures defensively:

- `Register` / `ForgotPassword` / `Resend` wrap in `try/catch`, **never** surface delivery errors to the user (to avoid leaking account existence).

- `VerifyEmailModel.OnPostResend` exposes a generic "we couldn't send the email right now" when the user explicitly asked for a resend (that request isn't about existence leakage).

---

6. Diagnostic endpoints

Path	Purpose
`GET /__health/db`	JSON: `{ ok, products, database:"sqlite", file }`
`GET /Home/Error`	Friendly error page when unhandled exceptions bubble up in Production

---

Continue to `05_Deployment_and_Study.md`.

05 — Deployment, Study Plan, Risks, and Final Summary

1. Running locally

1.1 Pre-requisites

- **.NET 8 SDK** — `dotnet --version` should print `8.x`.
- Windows, macOS, or Linux.
- No separate DB server — SQLite is file-based (`pixelbit.db`).

1.2 Start with hot reload

```
# From the repo root, in a dedicated terminal window:
./scripts/start-dev.ps1
```

This runs `dotnet watch run --project "PixelAndBit.Web/PixelAndBit.Web.csproj"`. Keep the window open; stop with `Ctrl+C`.

- Dev binding: `http://0.0.0.0:5001` (LAN testing from your phone works).
- Browse: `http://localhost:5001`.
- Default seeded admin: `admin@pixelbit.jo` / `Admin@Pb2026!`.

1.3 Reset local DB

```
# Stop the app first if running.
Remove-Item PixelAndBit.Web/pixelbit.db -Force
Remove-Item PixelAndBit.Web/pixelbit.db-wal -ErrorAction SilentlyContinue
Remove-Item PixelAndBit.Web/pixelbit.db-shm -ErrorAction SilentlyContinue
# Next run will re-migrate and re-seed.
```

1.4 Add a new migration

```
cd PixelAndBit.Web
dotnet ef migrations add <Name> --project ../PixelAndBit.Infrastructure --startup-project .
dotnet ef database update --project ../PixelAndBit.Infrastructure --startup-project .
```

Migrations live in `PixelAndBit.Infrastructure/Migrations/`. Program startup also auto-applies them on first request.

2. Publishing

2.1 Framework-dependent publish

```
dotnet publish PixelAndBit.Web/PixelAndBit.Web.csproj -c Release -o publish
```

This writes the app + assets to `./publish/`. You can copy that folder to any Linux/Windows host with the matching .NET 8 runtime installed.

2.2 Self-contained publish (no .NET install required on the host)

```
dotnet publish PixelAndBit.Web/PixelAndBit.Web.csproj -c Release -r linux-x64 --self-contained true -o publish-linux
```

2.3 What ends up in publish?

- `PixelAndBit.Web.dll` + dependencies
- `wwwroot/` assets (favicon, logo, `pixelbit.css`, `site.js`, libs)
- `appsettings*.json` — provide `appsettings.Production.json` on the server with real SMTP credentials.
- `pixelbit.db` — usually **do not** ship; let the app create a fresh file on first run (it'll migrate and — only in Development — seed).

---

3. Linux host pattern (Kestrel + systemd + Nginx)

This project targets a reverse-proxy deployment. The Kestrel port in Production defaults to `127.0.0.1:5001` (see `Program.cs`), so it's only reachable via the reverse proxy.

> There is no `.service`, `nginx.conf`, or `Dockerfile` shipped in the repo. The snippets below are the recommended shape based on the project's runtime behavior.

3.1 systemd unit (example)

`/etc/systemd/system/pixelbit.service`

```
[Unit]
Description=Pixel & Bit web app (ASP.NET Core 8)
After=network.target

[Service]
WorkingDirectory=/var/www/pixelbit
ExecStart=/usr/bin/dotnet /var/www/pixelbit/PixelAndBit.Web.dll
Restart=always
RestartSec=10
SyslogIdentifier=pixelbit
User=www-data
Environment=ASPNETCORE_ENVIRONMENT=Production
# Explicit override (already the default in Program.cs):
Environment=ASPNETCORE_URLS=http://127.0.0.1:5001

# Secrets via env, not appsettings.json:
Environment=Smtp__Host=smtp.gmail.com
Environment=Smtp__Port=465
Environment=Smtp__EnableSsl=true
Environment=Smtp__Username=YOUR_USER
Environment=Smtp__Password=YOUR_APP_PASSWORD
Environment=Smtp__FromEmail=YOUR_USER
Environment=Smtp__FromName=Pixel & Bit

[Install]
WantedBy=multi-user.target
```

Operate:

```
sudo systemctl daemon-reload
sudo systemctl enable --now pixelbit
sudo systemctl status pixelbit
journalctl -u pixelbit -f
```

3.2 Nginx (TLS-terminating reverse proxy)

`/etc/nginx/sites-available/pixelbit`

```
server {
    listen 80;
    server_name pixelbit.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name pixelbit.example.com;

    ssl_certificate     /etc/letsencrypt/live/pixelbit.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/pixelbit.example.com/privkey.pem;

    # Big-enough for HTML email templates (just in case)
    client_max_body_size 10m;

    location / {
        proxy_pass         http://127.0.0.1:5001;
        proxy_http_version 1.1;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   Upgrade           $http_upgrade;   # WebSockets (future-proof)
        proxy_set_header   Connection        keep-alive;
    }
}
```

Enable + reload:

```
sudo ln -s /etc/nginx/sites-available/pixelbit /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

3.3 Forwarded headers (optional but recommended)

If you rely on `HttpContext.Request.Scheme == "https"` or `RemoteIpAddress` in code or logging, add to `Program.cs` before `UseAuthentication`:

```
app.UseForwardedHeaders(new ForwardedHeadersOptions
{
    ForwardedHeaders = ForwardedHeaders.XForwardedFor | ForwardedHeaders.XForwardedProto
});
```

This is **not** currently wired in `Program.cs` — consider adding it when you move to production.

3.4 Persistence of SQLite

- Keep `pixelbit.db` in a writable directory owned by the service user (e.g., `/var/www/pixelbit/`).
- Back it up periodically (`sqlite3 pixelbit.db ".backup /var/backups/pixelbit-$(date +%F).db"`).
- Prefer keeping WAL journal on (`pragma journal_mode = wal;` default).

---

4. Windows hosting (optional)

You can also host on Windows via **IIS + AspNetCoreModuleV2**:

- Publish the app.
- Configure an IIS site pointing to the publish folder.
- Install the **.NET 8 Hosting Bundle**.
- Make the app pool run with a user that can write the SQLite file.
- Set environment variables via the IIS Configuration Editor (same names as above, e.g. `ASPNETCORE_ENVIRONMENT=Production`).

---

5. Study plan — how to read this project

A step-by-step order that matches how the app is wired.

Step 1 — Orient yourself

1. Open `PixelAndBit.sln` to see the four projects.
2. Read **`docs/01_Overview.md`** — gives you the mental model.
3. Open `PixelAndBit.Web/Program.cs` and skim. Spot:

- SQLite path override
- Identity config (password + `RequireConfirmedAccount`)
- Session + distributed cache
- Cookie events redirecting API calls to 401/403
- `EnsureDatabaseInitializedAsync` pattern
- Routing order

Step 2 — Domain & Application

1. `PixelAndBit.Domain/Entities/*.cs` — 7 files, ~100 lines each.
2. `PixelAndBit.Domain/Enums/*.cs` — 4 tiny enums.
3. `PixelAndBit.Application/Interfaces/*.cs` — the service contracts.

Step 3 — Infrastructure

1. `Data/AppDbContext.cs` — inherits `IdentityDbContext<IdentityUser>`.
2. `Data/Configurations/*.cs` — learn how Fluent API shapes each table.
3. `Migrations/20260418124033_InitialSqlite.cs` — how it becomes SQL.
4. `Data/DbSeeder.cs` — roles + admin + default products.
5. `Data/ProductService.cs`, `BookingService.cs`, `CartService.cs`, `AppointmentService.cs` — implementations.
6. `Email/SmtpEmailSender.cs` + `NullEmailSender.cs`.

Step 4 — Web layer (the fun part)

1. `Controllers/HomeController.cs` (simplest).

2. `Controllers/StoreController.cs` (search + catalog).
3. `Controllers/BookingController.cs` (booking, tracking, admin status change).
4. `Controllers/CartController.cs` (session cart + checkout).
5. `Controllers/ProfileController.cs` (authenticated user history).
6. `Controllers/AdminController.cs` (dashboard math: `SumAsync<double?>` cast to decimal — read the comment above it).
7. `Controllers/AdminUsersApiController.cs` (JSON API).
8. `Controllers/LanguageController.cs` (culture cookie).

Step 5 — Identity

1. `Areas/Identity/Pages/Account/Login.cshtml(.cs)` — shared login.
2. `Register.cshtml(.cs)` + `VerifyEmail.cshtml(.cs)` — 5-digit email verify flow.
3. `PasswordResetFlow.cs` — session keys + helpers.
4. `ForgotPassword / VerifyResetCode / ResetPassword` — Steps 1→2→3.
5. `Logout.cshtml.cs` — POST-only safety.

Step 6 — UI

1. `Views/Shared/_Layout.cshtml` — the backbone.
2. `Views/Shared/_Navbar.cshtml` — active-link tracking, role-aware, cart badge, EN/AR toggle, offcanvas.
3. `Views/Home/Index.cshtml` — the biggest homepage view.
4. `Views/Booking/*`, `Views/Store/Index.cshtml`, `Views/Cart/*`, `Views/Admin/*`, `Views/Profile/*`.
5. `wwwroot/css/pixelbit.css` — search for the groups documented in `03_Frontend.md`.
6. `wwwroot/js/site.js` — short and worth reading end to end.

Step 7 — Localization

1. `Resources/SharedResource.cs` (the marker class).
2. `Resources/SharedResource.resx` / `.ar.resx` — same keys in both files.
3. `LanguageController.cs` + the lang forms in the navbar.

Step 8 — End-to-end smoke test

1. Register a new account; copy the code from the inbox; verify; you're logged in.
2. Book a repair; note the `PB-2026-XXXX` ticket; sign out; track the ticket anonymously.
3. Add a store item, check the badge updates (AJAX), confirm checkout.
4. Sign in as `admin@pixelbit.jo`; move a booking to `ReadyForPickup`; mark an order `Completed`; watch the dashboard numbers move.
5. Do a "Forgot password" flow; ensure the 3-step gating works (direct-hit Step 2 / Step 3 URLs redirect you back correctly).

---

6. Problems / Risks / Improvements

This section is honest about what could bite you.

6.1 Security

- **Secrets in source control** — `appsettings.json` contains a real Gmail app password. Remove it, use environment variables / `dotnet user-secrets` in dev, and a secrets manager in prod.
- **Seeded admin password** — `Admin@Pb2026!` is hardcoded. Change after first deploy, or gate seeding behind a first-run flag.
- **Password policy** is relaxed (6 chars, no digits). Tighten for production: `RequireDigit = true`, `RequireNonAlphanumeric = true`, and consider raising `RequiredLength` to 8.
- **No HTTPS redirection** is enabled. Add `app.UseHttpsRedirection()` if the app is ever exposed directly. Behind Nginx with HSTS, this is already covered.
- **Forwarded headers** not wired — logging IPs and issuing secure cookies behind Nginx relies on `X-Forwarded-*`.
- **CSRF on AJAX** works because of `window.__pb.csrf`. Anyone adding new fetch calls must remember to forward the `RequestVerificationToken` header.
- **Role claims cache** — when you add/remove a user from a role, existing cookie sessions keep the old role until they refresh / log out. Consider `UpdateSecurityStampAsync` after role changes too.

6.2 Data & reliability

- **SQLite** is fine for a single instance but not ideal for scale-out. Behavior under very high concurrency (and especially inside OneDrive) is risky — keep the `pixelbit.db` file on a local volume in production.
- **No FK from `UserId` string columns to `AspNetUsers.Id`** — by design, to allow guest rows. But it also means deleting a user does **not** remove their historical bookings/orders. That might be desirable (audit) or not (privacy).
- **No transaction in `Cart.Confirm`** — inserting `Order` and clearing the session cart are not atomic. If the app crashes between them, the cart persists but the order is already saved. Low risk but easy to harden with a transaction.

6.3 Email

- Password in `appsettings.json` — see secrets note above.
- `SmtpEmailSender` hardcodes `client.ConnectAsync(host, port, useSsl: true)` (implicit SSL). If you ever switch to `starttls` on port 587, this line needs changing.
- Stdout `Console.WriteLine("SMTP DEBUG …")` leaks the From address and auth username to logs. Consider a log-level guard or structured logging.

6.4 Performance

- `StoreController` filters in memory after `ToList` — fine for the current 5-product catalog, not fine at 1k+. Move the `Contains` filter into the query for scale.
- `AdminController.Index` sums over `(double?)` to dodge the EF Core SQLite decimal-sum limitation. Comment is in the code; document it if the provider changes.
- Homepage loads 4 font families and several Google Fonts subsets. Consider local hosting or `font-display: swap` optimization.

6.5 UX / Accessibility

- Buttons/inputs use `pb-input` / `pb-btn-glow`; focus-visible states exist but verify contrast on light OS themes.

- Color-only cues in status badges — pair with icon/text for accessibility.
- Provide `aria-live` on the cart badge so screen readers announce updates.

6.6 Feature gaps

- `AppointmentService` — `GetAvailableDatesAsync` / `GetAvailableSlotsAsync` are stubs returning empty. Slot-based booking UI isn't wired to the user-facing site yet.
- No admin UI to manually confirm someone's email or resend a code.
- No paginated/search-able admin `Users` view (the JSON API just returns everything).
- Password strength meter, "show password" toggle, and 2FA enrollment are all missing.

6.7 Testability

- No test project exists. The cleanest candidates to add are:
- `BookingServiceTests` — phone regex, ticket generation collisions, status transitions.
- `CartServiceTests` — add/remove/clear semantics over a fake session.
- `PasswordResetFlowTests` — hashing, cooldown math, session gating.

6.8 Operations

- Dev server prefers `0.0.0.0:5001`; production prefers `127.0.0.1:5001`. Document this clearly in ops runbooks — several engineers stumble on "why can't I reach it externally" on production without a reverse proxy.
- Add a CI workflow running `dotnet build` + `dotnet test` before merging to main.
- A `Dockerfile` + `docker-compose.yml` would make onboarding faster; SQLite volume mount is straightforward.

---

7. Final summary

- The app is a **well-layered ASP.NET Core 8 application** following a clean architecture with four projects: `Domain`, `Application`, `Infrastructure`, `Web`.
- It combines **MVC controllers** for business pages with **Razor Pages (Identity)** for auth, under a **single shared layout** — the UI looks consistent everywhere.
- **Business features**: store with session cart + guest checkout; repair booking with Jordanian-phone validation and a collision-resistant ticket generator; public ticket tracking; admin dashboard with revenue math and status transitions.
- **Auth**: ASP.NET Core Identity with a **custom 5-digit email verification** code (persisted, hashed, rate-limited) and a **session-backed multi-step forgot-password flow** that reuses Identity's own reset token for the actual password mutation.
- **I18n**: cookie-based culture, `en` / `ar` resources, full RTL via Bootstrap RTL + `.pb-rtl` overrides.
- **Styling**: one large `pixelbit.css` defines the premium dark theme, glassmorphism surfaces, gradients, and motion system; one small `site.js` drives fade-ins, parallax variables, device chips, and cart AJAX.
- **Deployment**: SQLite on a local volume; Kestrel bound to loopback; Nginx as TLS-terminating reverse proxy; systemd for lifecycle.
- **Improvements worth prioritising**: pull secrets out of source, rotate the seeded admin password, tighten password policy for production, add forwarded-headers + HTTPS redirection, and add a small test project.

Documentation file created at: `/docs/PixelAndBit_Documentation.md`

Individual sections are at:

- `/docs/01_Overview.md`
- `/docs/02_Backend.md`
- `/docs/03_Frontend.md`
- `/docs/04_Database_Auth_Config.md`
- `/docs/05_Deployment_and_Study.md`